Lab 10

1. Introduction

In modern operating systems, processes often access data stored in memory in units called pages. When a process tries to access a page that is not currently in memory, a page fault occurs. Handling page faults efficiently is critical because excessive page faults can significantly slow down system performance.

When the physical memory is full, the operating system must decide which page to remove to make space for the new one. This decision is handled by page replacement algorithms.

Two widely used page replacement strategies are:

1. **FIFO (First-In-First-Out):**
    - This strategy evicts the page that has been in memory the longest, regardless of whether it has been used recently.
2. **LRU (Least Recently Used):**
    - This strategy evicts the page that has not been used for the longest time, taking into account the actual usage pattern of pages.

The goal of this lab exercise is to implement the LRU algorithm, simulate both FIFO and LRU on the same workload, and analyze their performance by comparing the total number of page faults.

2. Algorithm Explanations

2.1 FIFO (First-In-First-Out)

- Memory is treated as a queue.
- When a page fault occurs:
    - If memory is not full → add page to the end.
    - If memory is full → remove the front page (the oldest).
- Simple but does not consider usage patterns.
- Can cause Belady's anomaly (more frames → more page faults in some cases).

2.2 LRU (Least Recently Used)

- Memory maintains a list of recency.
- Each page access:
    - If page is in memory → move it to the most recent end.
    - If page fault occurs:
        - If memory has space → add page at MRU end.
        - If memory full → remove least recently used page (front) and insert new page at MRU end.

- Models temporal locality of processes (recently used pages are likely to be used again).
- Usually performs better than FIFO for real workloads.

FIFO:

```
hp@TF-COMP-786:~/my_project$ cat > page_replacement.py
from collections import deque
def fifo_page_replacement(page_reference_string, num_frames):
    print("--- Starting FIFO Simulation ---")
    print(f"Page Reference String: {page_reference_string}")
    print(f"Number of Frames: {num_frames}\n")
    frames = [] # Represents the physical memory frames
    page_faults = 0
    # Use a deque as a queue to easily track the first-in page
    page_queue = deque()
    for page in page_reference_string:
    # Check if the page is already in a frame (a "hit")
        if page in frames:
            status = "Hit"
        # If not in a frame, a "fault" occurs
        else:
            page_faults += 1
            status = "Fault"
        # If there is space in memory, simply add the page
            if len(frames) < num_frames:
                frames.append(page)
                page_queue.append(page)
        # If memory is full, replace the oldest page
            else:
                page_to_replace = page_queue.popleft() # Get the oldest page
                frame_index = frames.index(page_to_replace)
                frames[frame_index] = page # Replace it with the new page
                page_queue.append(page) # Add new page to the queue
    # Print the state at each step for clarity
        print(f"Request: {page}, Frames: {list(frames)}, Status: {status}")
    print("\n--- FIFO Simulation Complete ---")
    print(f"Total Page Faults: {page_faults}")
    return page_faults
```

LRU:

```
def lru_page_replacement(page_reference_string, num_frames):
    memory = []
    page_faults = 0

    print("\n--- LRU Page Replacement Simulation ---")
    print(f"Reference String: {page_reference_string}")
    print(f"Number of Frames: {num_frames}\n")

    for page in page_reference_string:
        if page in memory:
            memory.remove(page)
            memory.append(page)
            hit_or_fault = "Hit"
        else:
            page_faults += 1
            if len(memory) < num_frames:
                memory.append(page)
            else:
                memory.pop(0)
                memory.append(page)
            hit_or_fault = "Fault"

        print(f"Request: {page}, Frames: {memory}, Status: {hit_or_fault}")

    print(f"\nTotal Page Faults: {page_faults}")
    return page_faults


if __name__ == "__main__":
# Define a sample workload
    reference_string = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]
    frame_count = 3
    # Run the FIFO simulation
    fifo_faults = fifo_page_replacement(reference_string, frame_count)
    lru_faults =lru_page_replacement(reference_string, frame_count)

    print("\n===== Summary of Page Replacement Algorithms =====")
    print("Algorithm | Total Page Faults")
    print("-----------------------------")
    print(f"FIFO      | {fifo_faults}")
    print(f"LRU       | {lru_faults}")
```

```
hp@TF-COMP-786:~/my_project$ python3 page_replacement.py
--- Starting FIFO Simulation ---
Page Reference String: [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]
Number of Frames: 3

Request: 7, Frames: [7], Status: Fault
Request: 0, Frames: [7, 0], Status: Fault
Request: 1, Frames: [7, 0, 1], Status: Fault
Request: 2, Frames: [2, 0, 1], Status: Fault
Request: 0, Frames: [2, 0, 1], Status: Hit
Request: 3, Frames: [2, 3, 1], Status: Fault
Request: 0, Frames: [2, 3, 0], Status: Fault
Request: 4, Frames: [4, 3, 0], Status: Fault
Request: 2, Frames: [4, 2, 0], Status: Fault
Request: 3, Frames: [4, 2, 3], Status: Fault
Request: 0, Frames: [0, 2, 3], Status: Fault
Request: 3, Frames: [0, 2, 3], Status: Hit
Request: 2, Frames: [0, 2, 3], Status: Hit
Request: 1, Frames: [0, 1, 3], Status: Fault
Request: 2, Frames: [0, 1, 2], Status: Fault
Request: 0, Frames: [0, 1, 2], Status: Hit
Request: 1, Frames: [0, 1, 2], Status: Hit
Request: 7, Frames: [7, 1, 2], Status: Fault
Request: 0, Frames: [7, 0, 2], Status: Fault
Request: 1, Frames: [7, 0, 1], Status: Fault

--- FIFO Simulation Complete ---
Total Page Faults: 15

--- LRU Page Replacement Simulation ---
Reference String: [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]
Number of Frames: 3

Request: 7, Frames: [7], Status: Fault
Request: 0, Frames: [7, 0], Status: Fault
Request: 1, Frames: [7, 0, 1], Status: Fault
Request: 2, Frames: [0, 1, 2], Status: Fault
Request: 0, Frames: [1, 2, 0], Status: Hit
Request: 3, Frames: [2, 0, 3], Status: Fault
Request: 0, Frames: [2, 3, 0], Status: Hit
Request: 4, Frames: [3, 0, 4], Status: Fault
Request: 2, Frames: [0, 4, 2], Status: Fault
Request: 3, Frames: [4, 2, 3], Status: Fault
Request: 0, Frames: [2, 3, 0], Status: Fault
Request: 3, Frames: [2, 0, 3], Status: Hit
Request: 2, Frames: [0, 3, 2], Status: Hit
Request: 1, Frames: [3, 2, 1], Status: Fault
Request: 2, Frames: [3, 1, 2], Status: Hit
Request: 0, Frames: [1, 2, 0], Status: Fault
Request: 1, Frames: [2, 0, 1], Status: Hit
Request: 7, Frames: [0, 1, 7], Status: Fault
Request: 0, Frames: [1, 7, 0], Status: Hit
Request: 1, Frames: [7, 0, 1], Status: Hit

Total Page Faults: 12

===== Summary of Page Replacement Algorithms =====
Algorithm | Total Page Faults
------------------------------
FIFO      | 15
LRU       | 12
```

Which algorithm performed better?

- In this simulation, LRU performed better than FIFO, producing fewer page faults.
- The reason is that LRU keeps recently used pages in memory, reducing the chance of removing a page that will be needed again soon.

Why this difference occurs

- **FIFO** only considers the arrival order of pages, not how frequently or recently they are accessed.
- **LRU** accounts for recency of use, meaning pages that are being actively used stay in memory longer.

This simple change significantly reduces unnecessary evictions in workloads with temporal locality.